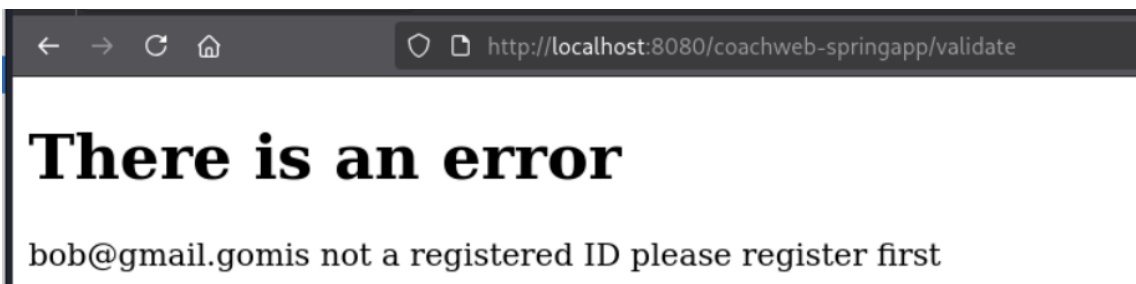
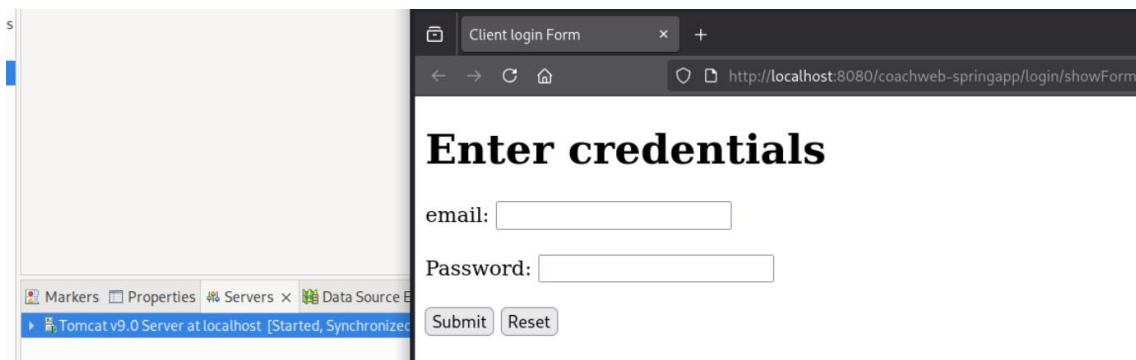
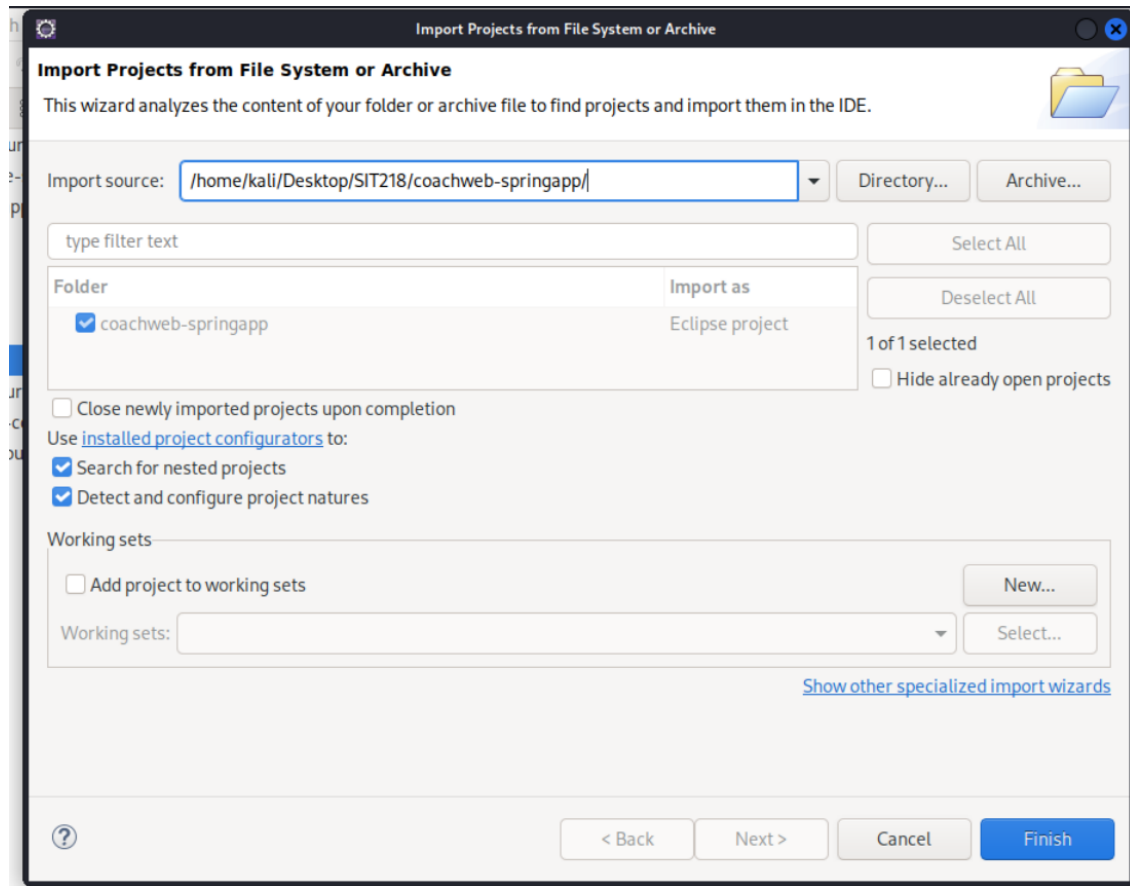


Task 3.2C

Setup



Task 1

The vulnerability here is that it directly appends the user input to the value that will be sent to the web browser as a response from the web server. There is no validation, no sanitization, to regex substitution or any other approach to replace dangerous characters, no escaping, no output encoding, just nothing and directly injecting the users input here. This is extremely dangerous and will introduce an XSS vulnerability.

```
Login.java *LoginController.java x
21
22 @PostMapping("/validate")
23 public String processLogin(@ModelAttribute("Login") Login login, Model model) {
24     if(login.getEmailID().equals("bob@gmail.com") & login.getPassword().equals("Password-1"))
25     {
26         model.addAttribute("greeting", "Welcome back");
27         return "result";
28     }
29     else if (!login.getEmailID().equals("bob@gmail.com"))
30     {
31
32         model.addAttribute("err", login.getEmailID() + "is not a registered ID please register first");
33         return "error";
34     }
35     else
36     {
37         model.addAttribute("err", "wrong credentials");
38         return "error";
39     }
}
```

```
Login.java *LoginController.java error.jsp x
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE HTML>
4 <html>
5 <head>
6   <title>Getting Started: Handling Form Submission</title>
7   <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
8 </head>
9 <body>
10  <h1>There is an error</h1>
11  <p> ${err} </p>
12
13 </body>
14 </html>
```

Task 2

Payload 1

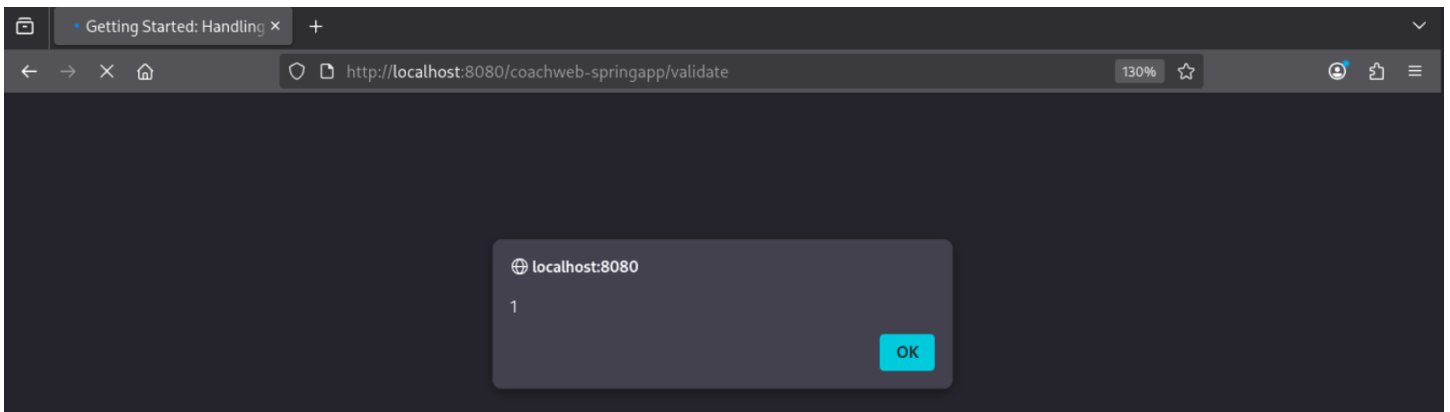
This is one of the most simplest and common PoC used when it comes to basic XSS – and here, we just use JavaScript's built in alert function, which is basically supported by every device / browser out there, including the smart systems in Teslas.

http://localhost:8080/coachweb-springapp/login/showForm

Enter credentials

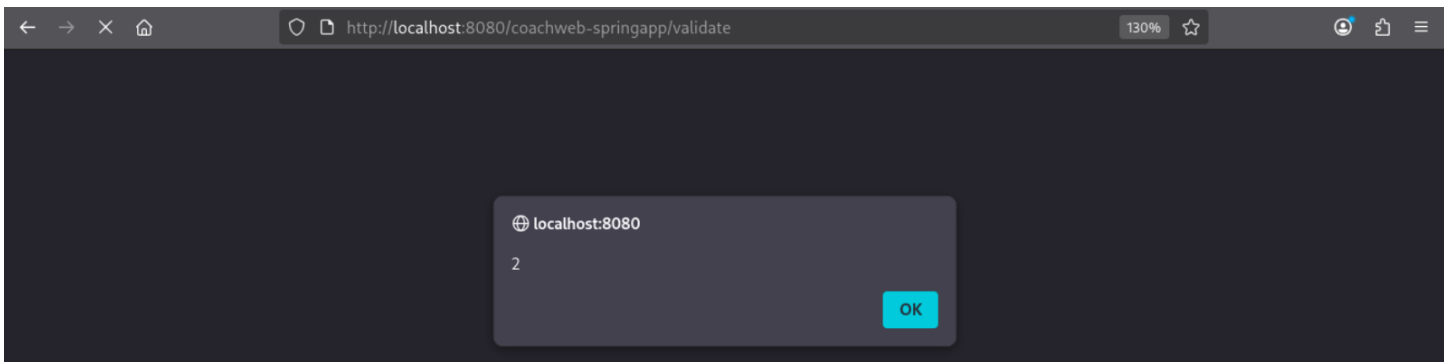
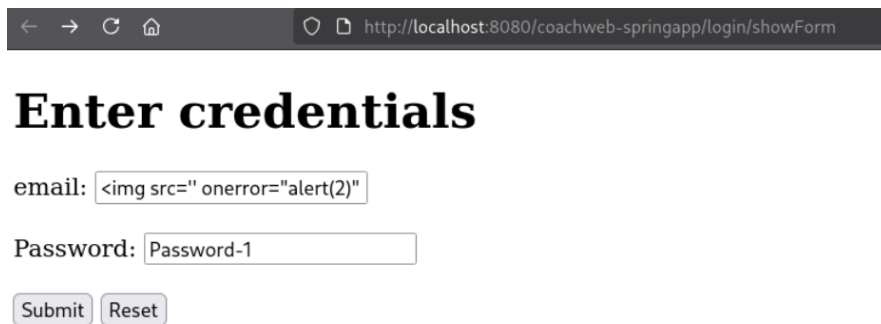
email:

Password:



Payload 2

Here, we try to load an image with the img tag. However, we don't properly set the source of the image to load it from but we set the onerror event to execute javascript. Since the image won't load, the onalert event will trigger and execute the javascript mentioned there. This is zero click and will execute the moment this img tag will get rendered in the users browser.

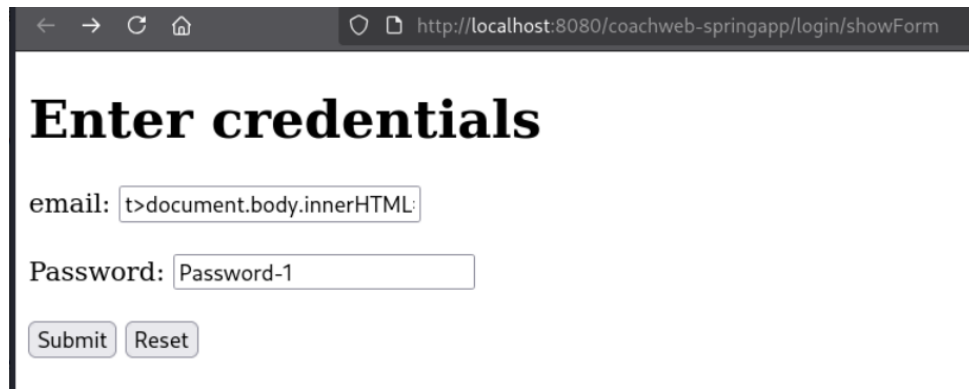


Payload 3

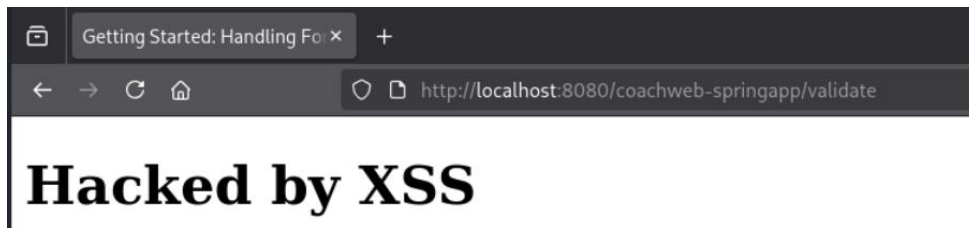
Here, instead of just doing a simple alert(), I'm using document.body.innerHTML to replace the entire content of the page's body with a level one header. The payload being used here is:

```
<script> document.body.innerHTML = "<h1>Hacked by XSS</h1>"; </script>
```

Here, the document.body will select the entire body of the DOM and setting the .innerHTML attribute will completely replace whatever is already there with the contents we set it to.



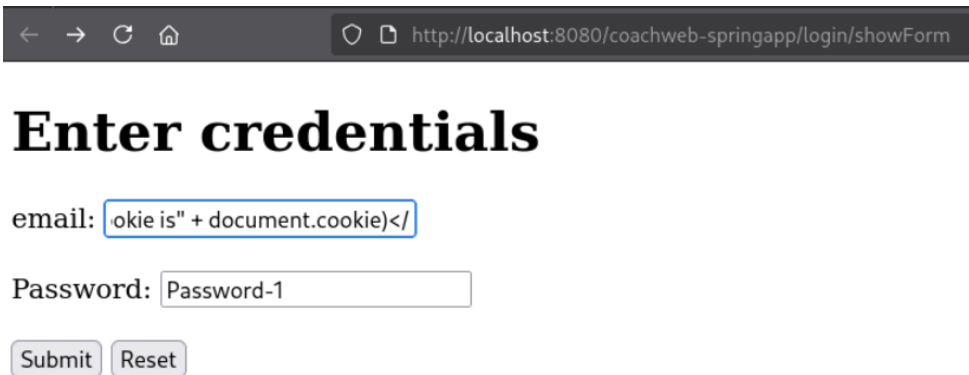
A screenshot of a web browser window. The address bar shows `http://localhost:8080/coachweb-springapp/login/showForm`. The page has a dark background with the title "Enter credentials" in large white text. Below the title, there are two input fields: "email:" and "Password:". The "email:" field contains the text `<document.body.innerHTML`. The "Password:" field contains the text "Password-1". At the bottom of the form, there are two buttons: "Submit" and "Reset".



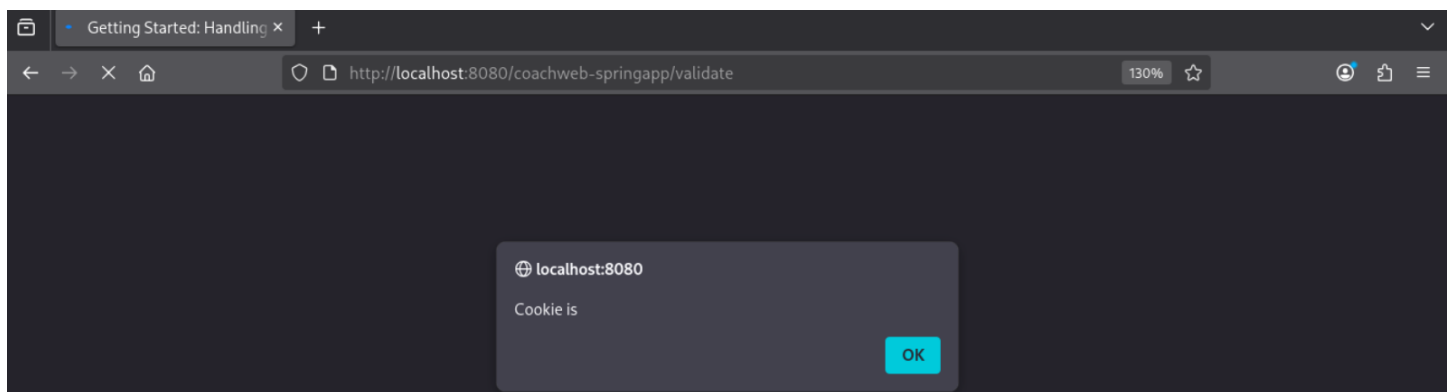
A screenshot of a web browser window. The address bar shows `http://localhost:8080/coachweb-springapp/validate`. The page has a dark background with the title "Hacked by XSS" in large white text.

Payload 4

Here, we attempt to access the non HttpOnly cookies stored in the browser for this web page by slightly modifying the first payload. However, no cookie was accessible in this case. Also, do note that HttpOnly cookies cannot be accessed this way. Therefore, it's always good to set that attribute for



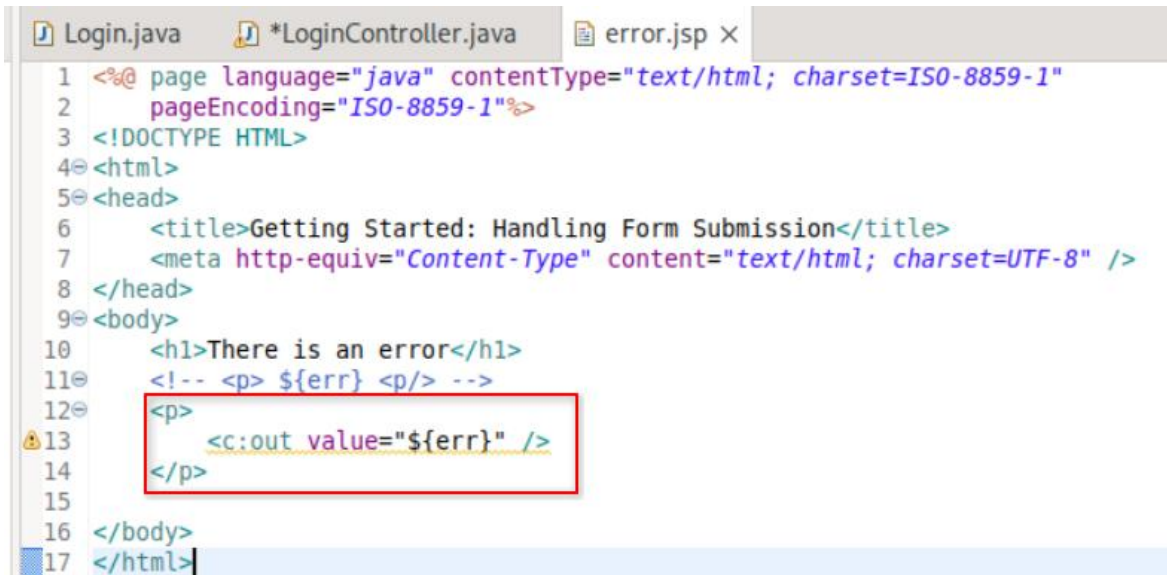
A screenshot of a web browser window. The address bar shows `http://localhost:8080/coachweb-springapp/login/showForm`. The page has a dark background with the title "Enter credentials" in large white text. Below the title, there are two input fields: "email:" and "Password:". The "email:" field contains the text `Cookie is" + document.cookie)</`. The "Password:" field contains the text "Password-1". At the bottom of the form, there are two buttons: "Submit" and "Reset".



Task 3

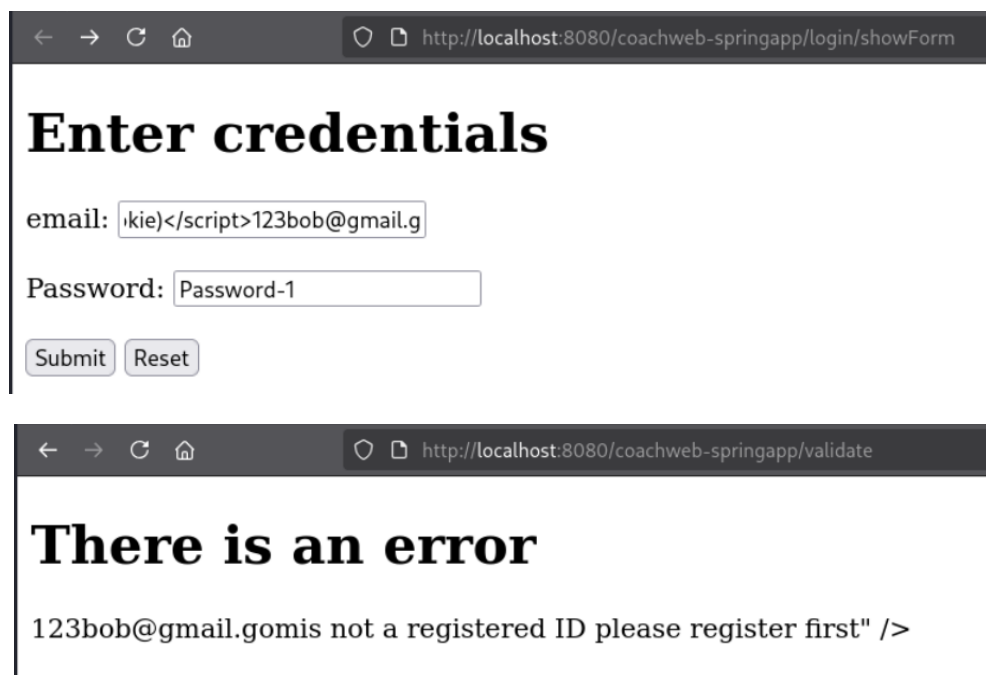
Following the task's instructions, you can either play around with the LoginController.java file and simply hard code an error message without dynamically appending the user input to the response being returned directly or sanitize it there.

However, a better way to do that would be to use a `<c:out ... />` tag in JSP. This will sanitize and escape any dangerous characters for you, automatically and is actually the recommended approach to display untrusted values^[1].

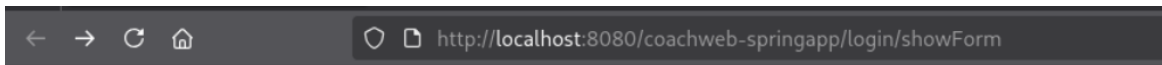


```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE HTML>
4 <html>
5 <head>
6   <title>Getting Started: Handling Form Submission</title>
7   <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
8 </head>
9 <body>
10  <h1>There is an error</h1>
11  <!-- <p> ${err} <p/> -->
12  <p>
13    <c:out value="${err}" />
14  </p>
15
16 </body>
17 </html>
```

I then tested it with a payload:



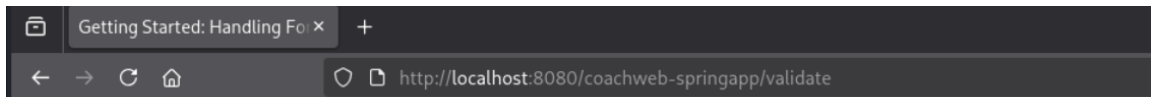
That worked. I then tested the third payload and it worked fine too.



Enter credentials

email:

Password:



There is an error

Hacked by XSS"123bob@gmail.gomis not a registered ID please register first" />

In development and secure coding, we should not remove something that we don't know how to fix, instead, we should attempt to fix it.

References

[1] ict1991, "Why Are the Escape Characters Being Displayed in HTML?," Stack Overflow, Jan. 11, 2012. <https://stackoverflow.com/questions/8827896/why-are-the-escape-characters-being-displayed-in-html> (accessed July 28, 2026).